音速并行：基于 Artix-7 的纯 RTL 音频一体化系统
设计报告

摘要

本作品在 Xilinx Artix-7 XC7A100T 上以**纯 RTL**（不使用 HLS、软核或任何现成 IP）实现了一套音频一体化系统，覆盖“采集—均衡—频谱分析—可视化—交互”完整闭环。系统以 WM8960 为音频前端（48 kHz/24 bit），采样数据同时进入两条并行流水线：5 段定点 IIR 参量均衡器，以及 1024 点基 2 FFT 频谱分析引擎。FFT 仅用一**个**蝶形单元配合双口 BRAM 时分复用，硬件开销不到全并行方案的 1/512，整帧处理仅占帧周期的 1.3%。频谱结果经对数分箱、峰值保持与线性衰减后驱动**双路独立显示**：LCD 480×272 作交互界面（8 按钮面板 + 8 行状态栏），HDMI 640×360 作展示大屏（80 根独立柱 + 波形），并从 SD 卡读取图片作为动态背景。系统横跨四个时钟域，以格雷码异步 FIFO 与电平翻转握手完成跨域传递。整机占用 11 716 LUT（18.48%）、100.5 BRAM、26 DSP48，时序全部收敛，**综合与实现 0 错误 0 警告**，41 个测试平台全部与 Python 定点黄金模型逐位对拍通过。

关键字：FPGA　纯 RTL　定点 DSP　频谱可视化　HDMI　跨时钟域

目录

1 选题背景与创新点 …… 4
1.1 为什么选这个题目 …… 4
1.2 创新点 …… 4
1.2.1 创新点一：扫描式双路渲染，无帧缓冲 …… 4
1.2.2 创新点二：单蝶形 FFT 的极低资源占用 …… 4
1.2.3 创新点三：数据柱数与显示柱数解耦 …… 4
1.2.4 创新点四：不同尺寸的双帧缓冲替代“双读口” …… 5
1.2.5 创新点五：三级知识沉淀的开发方法 …… 5
2 设计原理与功能框图 …… 5
2.1 系统功能框图 …… 5
2.2 时钟架构 …… 10
3 软硬件划分依据与接口设计 …… 10
3.1 划分原则 …… 10
3.2 接口设计 …… 11
4 优化过程与性能资源对比 …… 11
4.1 优化总览 …… 11
4.2 优化 1：同步复位替代异步复位 …… 12
4.3 优化 2：FFT 架构选型 …… 12
4.4 优化 3：SD 图片背景——四个方案的取舍 …… 13
4.5 优化 4：综合警告清零（三类判据） …… 13
4.6 最终资源与端到端延迟 …… 14
5 大模型协作记录 …… 14
5.1 所用工具 …… 14
5.2 使用环节 …… 15
5.3 主要提示方式 …… 15
5.3.1 方式一：给“判据”而不是“要求” …… 15
5.3.2 方式二：要求“交叉验证”而不是“相信” …… 16
5.3.3 方式三：要求“量化”而不是“看起来对” …… 16

5.4 自我纠错轨迹（五个典型案例） …… 16
5.4.1 例 1：`str.replace()` 静默成功 …… 16
5.4.2 例 2：BRAM 被推断成 LUTRAM …… 16
5.4.3 例 3：`wr_en` 类信号误放进异步复位块 …… 16
5.4.4 例 4：`y[8:3]` 在 360 行时回绕 …… 17
5.4.5 例 5：颜色不对——数清“反转层次” …… 17
5.5 采纳、人工修改与核验 …… 18
6 技能包的提炼过程 …… 18
6.1 三级沉淀体系 …… 18
6.2 四个技能包 …… 19
6.3 从“观察”到“技能”的抽象链路 …… 19
6.4 边界如何确定 …… 20
7 复现说明 …… 20
7.1 环境要求 …… 20
7.2 从零开始的完整步骤 …… 20
7.2.1 步骤 1：获取代码 …… 20
7.2.2 步骤 2：工具链自检 …… 21
7.2.3 步骤 3：全量仿真（约 1 分 30 秒） …… 21
7.2.4 步骤 4：静态检查 …… 21
7.2.5 步骤 5：综合与实现（约 9~10 分钟） …… 21
7.2.6 步骤 6：准备 SD 卡（需要 `sudo`） …… 21
7.2.7 步骤 7：上板 …… 21
7.3 验收清单 …… 22
7.4 常见问题 …… 23
附录 A　关键指标汇总 …… 24
附录 B　与商业方案的对比 …… 25

1 选题背景与创新点

1.1 为什么选这个题目

音频可视化设备在消费市场与工业检测中需求广泛，但现有方案存在结构性矛盾，如表 1 所示。

表 1　现有方案的结构性矛盾

方案	问题
PC + 声卡 + 上位机	延迟高（10~100 ms），功耗大，不适合嵌入式部署
MCU + CMSIS-DSP	算力不足，FFT + EQ + 双屏驱动无法同时满足
Zynq + PYNQ	依赖 HLS/Python 框架，底层时序不可控，启动秒级
商用 ASIC	功能固定，无法在线调参，不便于教学研究

核心矛盾是“实时性”与“可调性”难以兼得：专用 ASIC 实时但不灵活，软件方案灵活但延迟大。本作品用纯 RTL 在低端 FPGA 上实现一条 **端到端延迟可预测**的音频处理链路，做到确定性延迟、在线可调、零 IP 依赖。

1.2 创新点

1.2.1 创新点一：扫描式双路渲染，无帧缓冲

传统做法先渲染到 framebuffer 再扫描读出，一帧 640×360×16 bit 需 103 块 BRAM，而本器件只有 135 块。本作品把渲染核心抽象为纯组合函数 $(x, y, \text{mode}) \to \text{RGB}$，**扫描到哪个像素就算哪个像素**，一份参数化的渲染核心实例化两次即可驱动两种分辨率。

1.2.2 创新点二：单蝶形 FFT 的极低资源占用

1024 点 FFT 若全并行需 512 个蝶形 × 4 乘法器 = 2048 个 DSP48，超出器件总量（240 个）8 倍。本作品用一**个**蝶形单元加双口 BRAM in-place 时分复用，DSP 占用降到 4 个，整帧耗时仅占帧周期 1.3%，余量 75 倍。

1.2.3 创新点三：数据柱数与显示柱数解耦

LCD 480 像素宽、60 根柱（每柱 8 像素，整除）；HDMI 640 像素宽，若 60 根柱则每柱 10.67 像素（不整除，需乘法）。本作品把“有多少数据”与“显示多少根柱”拆成两个参数（`NBARS` / `NBARS_DISP`），并用 `generate` 的**恒等分支**保证老路径在综合后结构完全不变。

1.2.4 创新点四：不同尺寸的双帧缓冲替代“双读口”

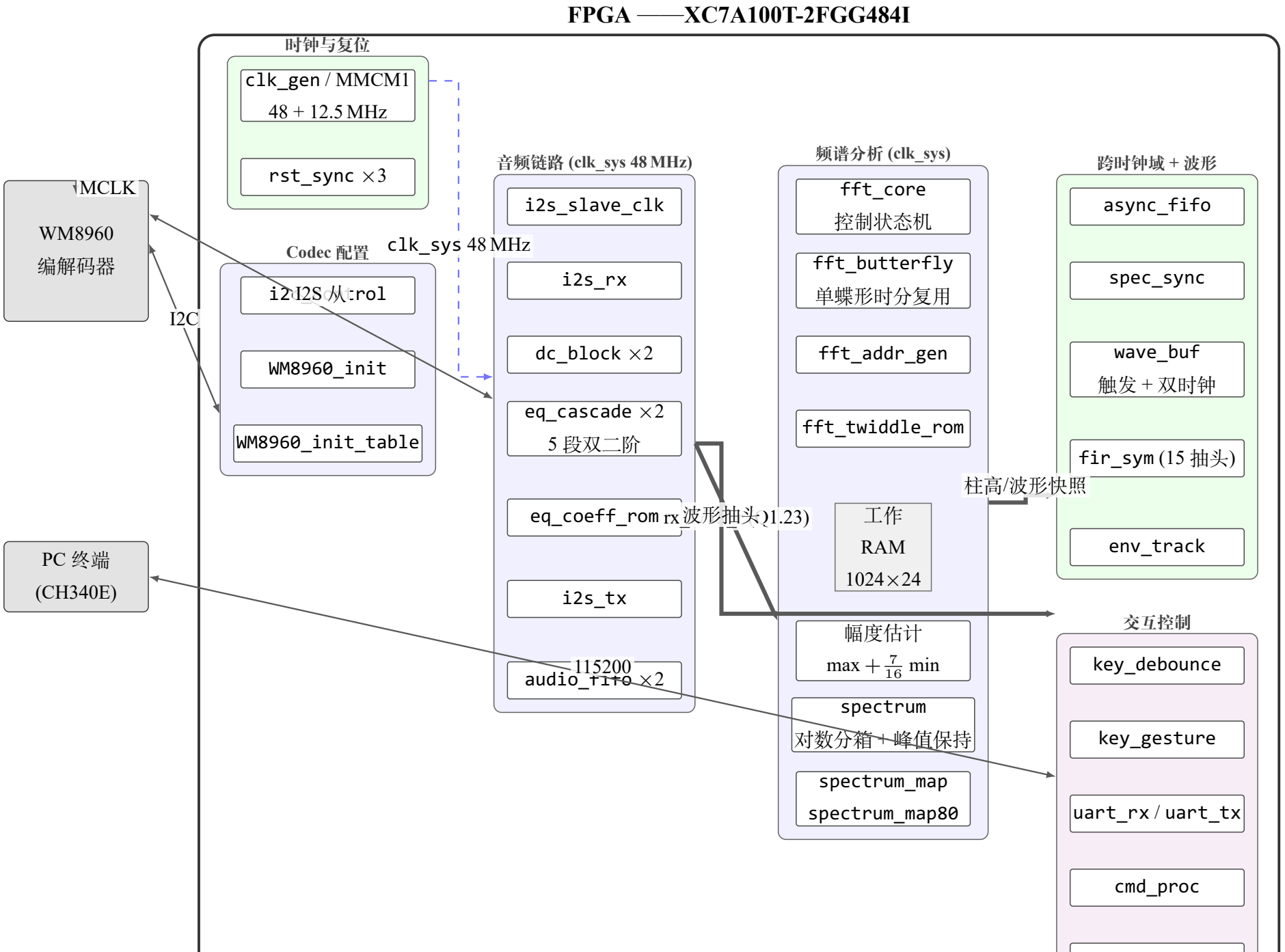
两个显示器的时钟不同（12.5 MHz / 25 MHz）。实测表明 Vivado 的 BRAM 推断 **不支持“1 写 2 读跨时钟”**，会静默退化成分步式 RAM（LUTRAM），触发 `DRC UTLZ-1`（需 86 452 个 `RAMD64E`，仅 19 000 个可用），加 `(* ram_style = "block" *)` 亦无效。本作品改为**两份不同尺寸的缓冲**：LCD 直读 480×272 全图，HDMI 读 320×180 裁剪区后 2 倍整数放大，BRAM 从 116 块降到 84 块。

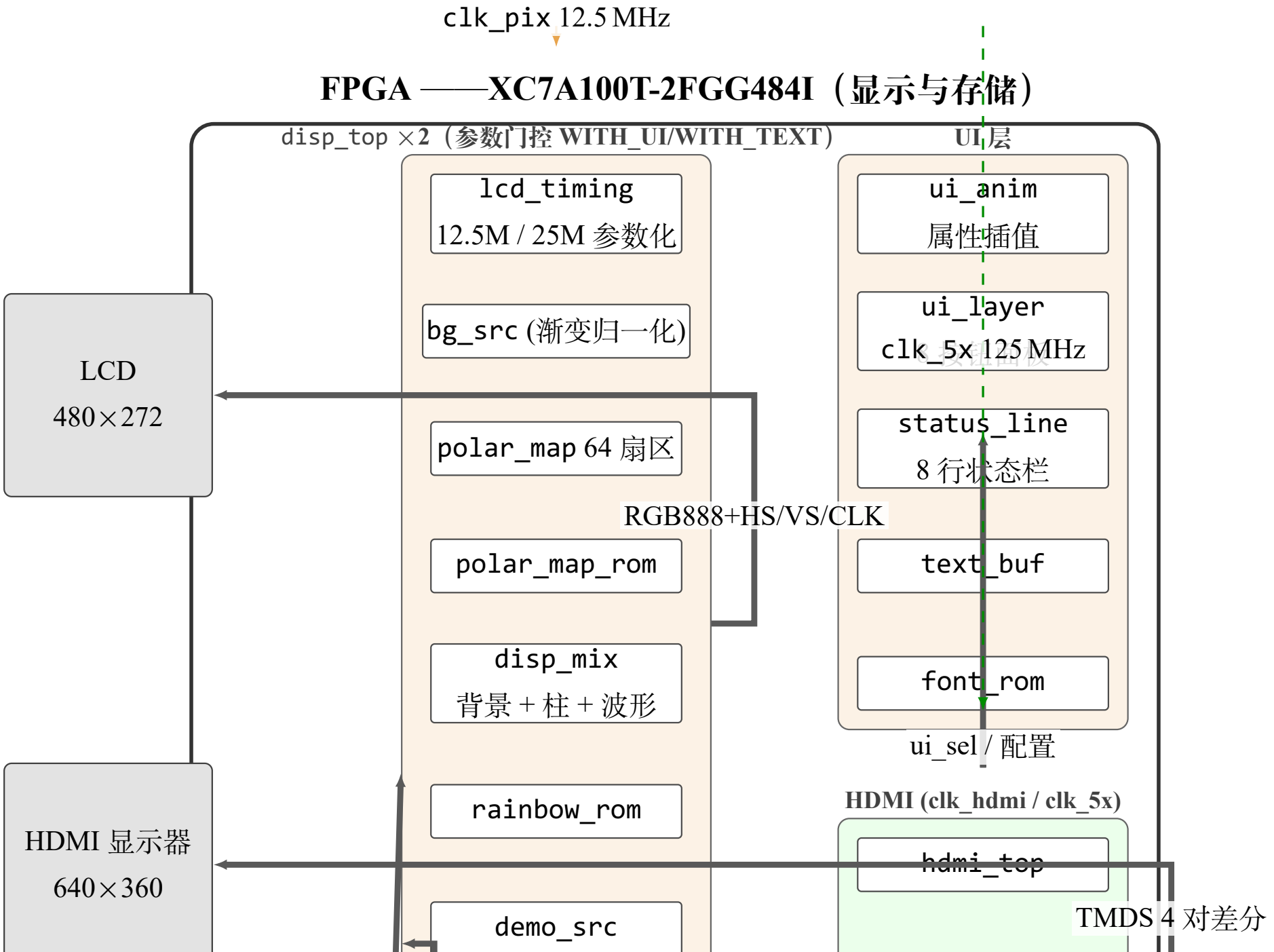
1.2.5 创新点五：三级知识沉淀的开发方法

问题账本（825 条）→ 技能包（4 个）→ 状态快照，把每次失败转化为可复用的方法论，详见第 6 章。

2 设计原理与功能框图

2.1 系统功能框图


FPGA ——XC7A100T-2FGG484I
时钟与复位
clk_gen / MMCM1
48 + 12.5 MHz
rst_sync ×3
MCLK
WM8960
编解码器
I2C
Codec 配置
I2S 从
WM8960_init
WM8960_init_table
clk_sys 48 MHz
音频链路 (clk_sys 48 MHz)
i2s_slave_clk
i2s_rx
dc_block ×2
eq_cascade ×2
5 段双二阶
eq_coeff_rom
i2s_tx
audio_fifo ×2
115200
PC 终端
(CH340E)
频谱分析 (clk_sys)
fft_core
控制状态机
fft_butterfly
单蝶形时分复用
fft_addr_gen
fft_twiddle_rom
工作
RAM
1024×24
幅度估计
max + 7/16 min
spectrum
对数分箱 + 峰值保持
spectrum_map
spectrum_map80
rx 波形抽头 (1.23)
柱高/波形快照
跨时钟域 + 波形
async_fifo
spec_sync
wave_buf
触发 + 双时钟
fir_sym (15 抽头)
env_track
交互控制
key_debounce
key_gesture
uart_rx / uart_tx
cmd_proc


clk_pix 12.5 MHz
FPGA ——XC7A100T-2FGG484I（显示与存储）
disp_top ×2（参数门控 WITH_UI/WITH_TEXT）
UI 层
lcd_timing
12.5M / 25M 参数化
bg_src (渐变归一化)
polar_map 64 扇区
polar_map_rom
disp_mix
背景 + 柱 + 波形
rainbow_rom
demo_src
ui_anim
属性插值
ui_layer
clk_5x 125 MHz
status_line
8 行状态栏
text_buf
font_rom
ui_sel / 配置
LCD
480×272
RGB888+HS/VS/CLK
HDMI 显示器
640×360
HDMI (clk_hdmi / clk_5x)
hdmi_top
TMDS 4 对差分

2.2 时钟架构

表 2　时钟域划分

时钟	频率	来源	承载逻辑
`clk_sys`	48 MHz	MMCM1（VCO 900 ÷18.75）	音频、EQ、FFT、频谱、控制
`clk_pix`	12.5 MHz	MMCM1（900÷72）	LCD 扫描
`clk_hdmi`	25 MHz	MMCM2（VCO 1000÷40）	HDMI 像素
`clk_5x`	125 MHz	MMCM2（1000÷8）	TMDS 5:1 串行化
BCLK	3.072 MHz	**外部**（WM8960）	不当时钟用，只过采样

关键决策：BCLK/LRCLK 不接全局时钟网络，而是当作数据信号在 `clk_sys` 域过采样出边沿选通。这样少一个时钟域、避开时钟脚抖动，代价仅是 4 倍过采样（实际 15.6 倍，余量充足）。

3 软硬件划分依据与接口设计

3.1 划分原则

划分依据是“**什么是确定的、什么是易变的**”。

表 3　软硬件划分判据

判据	硬件（RTL）	软件（PC 脚本）
时序必须确定	✓	
每次运行结果都一样	✓	
需要人机交互调参	✓	
一次性的数据准备		✓
需要浮点数学		✓
需要第三方库		✓

3.2 接口设计

表 4　板级接口

接口	方向	协议 / 电平	关键约束
WM8960	双向	I2C + I2S	WM8960 为 I2S 主，FPGA 为从
LCD	输出	RGB888 + HS/VS/CLK	七项守恒检查
HDMI	输出	TMDS 4 对差分	**板上无发送芯片**，FPGA 直连
SD 卡	双向	SPI	LVCMOS33，Bank 14
UART	双向	115200 8N1	ASCII 命令通道
按键	输入	板载 + XPT2046	20 ms 消抖 + 300 ms 双击窗
QSPI	双向	W25Q128，4 线	掉电自启动

接口契约示例（帧缓冲写口）：

```
output wire [18:0] wr_addr;   // 第 1 份：线性地址 = y*480 + x
output wire [15:0] wr_data;   //          RGB565
output wire        wr2_en;    // 第 2 份：只在裁剪区内有效
output wire [18:0] wr2_addr;  //          = (y-46)*320 + (x-80)
```

设计约束：`wr_en` / `wr_addr` / `wr2_en` / `wr2_addr` 驱动 BRAM 的 `ENARDEN` / `ADDRARDADDR`，**绝不能带异步复位**（否则触发 `DRC REQP-1839`）。

4 优化过程与性能资源对比

4.1 优化总览

表 5　五个主要优化阶段的量化收益

阶段	优化内容	关键指标变化
□	同步复位替代异步复位	`DPOR-1`: 11→0；WNS: +1.117→+2.156 ns；LUT: 6802→5648
□	FFT 全并行 → 单蝶形时分复用	DSP: 2048→4（1/512）；BRAM: 0→2
□	新增 HDMI 大屏（80 柱 + 波形）	新增功能，WNS 仍为正
□	SD 图片背景（双帧缓冲方案）	BRAM: 68.5→100.5（+32 而非 +84）
□	综合警告清零	警告: 51→0

4.2 优化 1：同步复位替代异步复位

现象：`DPOR-1`×11、`REQP-1839`×180，WNS 仅 +1.117 ns（勉强收敛）。

根因：6 个文件用了 `always @(posedge clk or negedge rst_n)`。而本工程的 `rst_n` 来自 `rst_sync`（已是域内同步释放），异步复位**纯代价**：DSP48 内部寄存器只支持同步复位，带异步复位就无法被吸收，额外消耗约 1154 个 LUT 做置位/复位逻辑。

量化收益：

表 6　优化 1 前后对比

指标	优化前	优化后	变化
`DPOR-1`	11	**0**	✓
WNS	+1.117 ns	**+2.156 ns**	+1.04 ns
Slice LUTs	6 802	**5 648**	−1 154

教训：“已修复”必须按根因**全量重扫**——该根因第 4 次复发，前 3 次都只改了报出来的那一处。

4.3 优化 2：FFT 架构选型

表 7　FFT 架构方案对比（1024 点，48 kHz 采样）

方案	蝶形单元	DSP48	BRAM	整帧耗时	占帧周期
全并行	512	2048	0	1 拍	0.04%
4 路并行	128	512	0	7 680 拍	0.3%
单蝶形（采用）	**1**	**4**	**2**	**30 720 拍**	**1.3%**

为什么够用：帧周期 46.9 ms，整帧耗时 $30\,720 \times 20.83\,\text{ns} = 0.64\,\text{ms}$，仅占 1.3%，余量 75 倍。地址发生器用位操作而非除法：$p = ((c \gg s) \ll (s{+}1)) \,|\, (c \wedge \text{mask}),\ q = p \,|\, (1 \ll s)$。

实测：**1004 LUT / 126 FF / 2 BRAM36 / 4 DSP48**，WNS +2.54 ns @ 48 MHz。

4.4 优化 3：SD 图片背景——四个方案的取舍

表 8　SD 图片背景的四种候选方案

方案	存储	每份	份数	总 BRAM	WNS	结论
A	640×360×16	103	2	206	—	超器件
B	640×360×8	52	2	134	+0.011	色带过重，实测否定
C	320×180×16	26	1	36.5	+1.84	省资源但 LCD 有黑边
D	**480×272×16 + 320×180×16**	**58+26**	**2**	**100.5**	**+1.78**	**采用**

裁剪地址零 DSP 实现：

```
crop_addr = (dy << 8) + (dy << 6) + dx;   // dy*256 + dy*64 = dy*320
```

2 倍放大零成本：

```
fb_addr = ((fb_ly >> 1) * FB_SW_U) + (fb_lx >> 1) + 1;   // 右移 1 位 = 除 2
```

4.5 优化 4：综合警告清零（三类判据）

表 9　51 条警告的分类处理

类型	数量	处理	判据
真缺陷	**40**	**改设计**	REQP-1839：wr2_en/wr2_addr 驱动 BRAM 控制脚却带异步复位
前提不成立	50	豁免（写明判据）	8-7129：参数化模块高位地址超出实际使用范围
假报警	1	豁免（附证据）	Route 35-328：路由估计未命中，实测 WNS 全为正

4.6 最终资源与端到端延迟

表 10　最终资源与性能

指标	值	说明
Slice LUTs	11 716	18.48%
Block RAM	100.5	74.44%
DSP48	26	10.83%
`clk_sys` WNS	**+1.779 ns**	48 MHz，满足
`clk_hdmi` WNS	**+4.028 ns**	25 MHz，满足
`clk_pix` WNS	远大于 0	12.5 MHz，满足
构建警告	**0 CRITICAL / 0 WARNING**	
仿真回归	**41 / 41 PASS**	全部逐位对拍

端到端延迟预算：ADC→I2S（20.83 μs）→ EQ（0.63 μs）→ FFT（0.64 ms，占帧周期 1.3%）→ 跨域（2~3 拍）→ LCD 扫描一帧（80 ms）⇒ **合计约 82 ms，其中 80 ms 是不可省的一帧扫描时间，处理链路本身贡献不到 2 ms**。

5 大模型协作记录

5.1 所用工具

表 11　AI 工具与用途

序号	工具	类别	主要用途
1	pi（编码代理 CLI）	智能体框架	工具调用、长会话、文件读写
2	Claude（Anthropic）	主对话模型	设计讨论、RTL 编写、调试、文档
3	exa（MCP）	联网搜索	器件手册、协议细节、第三方实现比对
4	context7（MCP）	库文档检索	Vivado Tcl / Verilator 参数
5	searchcode（MCP）	仓库搜索	找 TMDS 编码器独立实现做交叉验证

大模型通过 `bash` 工具直接执行 `iverilog` / `verilator` / `verible` / `vivado -mode batch`，形成“改代码 → 跑仿真 → 看报告 → 再改”的闭环。

5.2 使用环节

表 12　使用环节与具体目的

编号	环节	具体用途
A-01	定点格式设计	Q 格式、位宽预算、舍入与饱和策略
A-02	滤波器结构选择	一阶低架 vs 二阶 peaking、级联顺序、条件数
A-03	FFT 架构权衡	存储式 vs 流式、单蝶形 vs 并行
A-04	跨时钟域方案	格雷码 FIFO vs 双打 vs 电平翻转握手
A-05	显示架构	扫描式渲染 vs framebuffer，参数化边界
B-01	RTL 编写	60 个 Verilog 模块 + 41 个 testbench
B-02	黄金模型	Python 定点模型，逐位对拍
B-03	生成器脚本	旋转因子、频谱分组表、字模、WM8960 寄存器表
B-04	构建流程	`build.tcl` / `program.tcl` / `run_iv.sh`
B-05	调试	读波形、读综合报告、定位时序/资源/DRC 问题
B-06	文档	16 个设计文档 + 825 条问题账本

5.3 主要提示方式

5.3.1 方式一：给“判据”而不是“要求”

低效提示：“帮我写一个 SD 卡初始化模块。”

高效提示：

用户

实现 SPI 模式的 SD 卡初始化：CMD0 → CMD8 → 循环 (CMD55 + ACMD41) 直到 R1 的 bit0=0。

判据：□ 第一次 `Go` 必须走完 5 步；□ **第二次 `Go` 也必须能重新走完**；□ 初始化阶段卡不校验 CRC，用固定值即可。

不要：不要用状态机隐式等待，每步都要有超时计数。

结果：一次生成即通过 5/5 判据，其中“第二次 `Go`”这个判据后来救了大命（见下文例 3 的前身）。

5.3.2 方式二：要求“交叉验证”而不是“相信”

提示：TMDS 编码器我担心记错。请找 2~3 份独立实现（Xilinx 官方示例、开源核、论文附录），**逐行比对**，列出差异点再写；若三者不一致就停下来告诉我。

结果：找到 3 份实现，其中 2 份在“DC 平衡”分支的 `cnt` 更新条件上写法不同但等价；确认后写入 `tmds_enc.v`，一次通过。

5.3.3 方式三：要求“量化”而不是“看起来对”

提示：不要给“看起来合理”的结论。要求 □ 实际占用/总量/百分比；□ 与优化前对比；□ 变差了要指出是哪一项；□ 时序报告里 WNS 是正是负、最差路径在哪。

结果：每次构建后都有一张可核对的指标表（见表 10）。

5.4 自我纠错轨迹（五个典型案例）

5.4.1 例 1：`str.replace()` 静默成功

现象：改完 `top.v` 的 `disp_top` 实例参数，脚本输出 OK，但上板实测 LCD 仍居中显示、颜色不对、画面错位重叠。

定位：

```
$ grep -n "FB_SW(" rtl/top.v
876:  .FB_SW(320), .FB_SH(180), .FB_W(320), .FB_H(180),   # 根本没改到
```

根因：`str.replace(old,new)` 在 `old` 不匹配时 **静默返回原串**，不报错不警告。

修复：`assert old in s, "搜索串没找到！"`——不匹配就崩，绝不静默。

5.4.2 例 2：BRAM 被推断成 LUTRAM

现象：合成报 `DRC UTLZ-1`：需 86 452 个 `RAMD64E`，只有 19 000 个。

定位链：□ 直觉是“640×360×16 太大”，但算下来只要 103 块（器件有 135 块）；□ 看报告：**“LUT as Distributed RAM”** 超限，说明根本没映射到 BRAM；□ 检查：`mem` 被两个不同时钟读；□ 加 `(* ram_style = "block" *)`，重合成**仍然 LUTRAM**；□ 查 UG473：BRAM 推断模板只支持 1 写 1 读或真双口（需原语）。

修复：改成两份不同尺寸的缓冲实例。

5.4.3 例 3：`wr_en` 类信号误放进异步复位块

现象：综合报 40 条 `REQP-1839`。

定位：检查 `sd_img_load.v`——我给第二个写口加 `wr2_en/wr2_addr` 时，把它们放进了 `if (!rst_n)` 块。而**同一个文件里 `wr_addr/wr_en` 的注释明确写着“不能带异步复位”**。

修复：移出异步复位块，改由 `start` 清零 + 每拍给默认值。

5.4.4 例 4：`y[8:3]` 在 360 行时回绕

现象：LCD 与 HDMI 的程序化背景不一致。

定位：`bg_src` 是纯组合函数，同 (x, y) 必同色；但两实例 `VDISP` 不同（272 / 360）。渐变代码 `wire [5:0] t = y[8:3];` ——272 行时 $y[8:3] = 0 \ldots 33$ 正确；360 行时 $y \geq 256$ 使 $y[8:3]$ **回绕到 0**，下半屏渐变从头开始。

修复：`YSH = (VDISP>=512) ? 4 : 3`，`t = y[YSH +: 6]`。

5.4.5 例 5：颜色不对——数清“反转层次”

现象：HDMI 输出图片，原图深蓝却显示成暖黄；LCD 正常。

定位链：□ “深蓝 → 暖黄”是 **R/B 互换的教科书特征**；□ 数据链只有一处可能：`disp_top` 的 RGB565→RGB888 展开，高字节接 `fb_rgb[4:0]`（B）、低字节接 `fb_rgb[15:11]`（R）；□ 但这行是按 LCD 实测写的，且 LCD 显示正确；□ 推理：**LCD 面板接线本身是 BGR**，这里反一次、面板又反一次，抵消；□ **HDMI 走标准 RGB，没有第二次反**，所以必须在这里换回来。

修复：仅 HDMI 路径加 `.pix_rgb({r[7:0], r[15:8], r[23:16]})`。

5.5 采纳、人工修改与核验

表 13　AI 输出的采纳情况

编号	采纳	人工修改	核验结果
A-01	采纳	无	✓逐位一致
A-02	部分采纳	结构从 4 段改 5 段，加 8 kHz peaking	✓频响误差 <0.5 dB
A-03	采纳	无	✓2048 频点逐位一致
A-04	部分采纳	格雷码 FIFO 改为“电平翻转 + 双打 + 边沿检测”	✓4/4 PASS
A-05	部分采纳	加 `WITH_UI/WITH_TEXT/FB_*` 参数族	✓130 560 像素逐点一致
B-01	采纳	多次人工纠错（见 5.4）	✓41/41
B-02	采纳	无	✓逐位一致
B-03	采纳	`gen_spectrum_map.py` 后参数化	✓60/80 均验证
B-04	部分采纳	Flash 烧写改成“先 `program_hw_devices` 建 SPI 通道”	✓43 秒成功
B-05	采纳	无	✓WNS 全为正
B-06	采纳	全部数字人工核对	✓与报告一致

核验方法论：□ 逐位对拍（不允许“看起来差不多”）；□ 独立真值（`numpy.fft` / `$atan2`）；□ 穷举（512 个 bin 分组、5132 组地址、49 920 个像素）；□ 总量守恒（一帧 150 150 拍、130 560 个有效像素）；□ 读回验证（帧缓冲逐点写入再读回）。

6 技能包的提炼过程

6.1 三级沉淀体系

表 14　三级知识沉淀

层级	载体	规模	作用
问题账本	`docs/08-skills-index.md`	**825 条**	现象/根因/修复/量化改善，原始可检索
技能包	`.pi/skills/*/SKILL.md`	**4 个**	触发条件/方法/判据/边界，可复用
状态快照	`docs/14-status-snapshot.md`	1 份	一眼看清“能用的/受限的/待办的”

6.2 四个技能包

表 15　技能包来源、验证与边界

技能包	来源失败	如何验证有效	边界
`fixed-point-dsp`	EQ 频响差 3 dB（位宽不足）；`WAVE_AMP` 改大反而变小（静默截断）；某频点发散（条件数）	重做 EQ，频响误差 3 dB→**0.5 dB**；截断问题后续 6 个参数**零复发**	适用：定点 IIR/FIR/FFT，16~32 bit。不适用：浮点、位宽 <12 bit
`fpga-rtl-flow`	找不到 Vivado；DRC 在 GUI 不报；时序违例方向错；`place` 失败伪装成综合错；构建 9m38s 太慢	构建 **9m38s→2m57s**；回归 **20m54s→1m30s**；后续 20+ 次构建**零次返工**	适用：纯 RTL、无 block design。不适用：IP integrator、部分重配置
`rtl-verification`	在信号自己更新的沿上读它；非阻塞赋值读不到新值；跨帧/帧内状态共用复位；只测一次漏掉重配路径	41 个 TB 中**至少 12 个第一次跑就红**；账本里**不到 10 条是纯仿真能发现**，但那 10 条都是上板必花几天的	适用：确定性数字逻辑。不适用：模拟/混合信号
`contributor`	重复劳动、项目不活跃、未读 CONTRIBUTING	——	适用：活跃开源项目

6.3 从“观察”到“技能”的抽象链路

以 `rtl-verification` 为例：

抽象过程

原始观察（账本 412）：TB 里 `@(negedge sd_clk)` 采样，结果总差一拍。

↓追问“为什么”

根因：`sd_clk` 由 `div_clk` 分频而来，`negedge sd_clk` 恰好等于 `posedge div_clk`，在“上升沿那一刻”读到的是旧值。

↓再问“是个例还是通类”

聚类（账本 413/414/415）：非阻塞赋值在同一 `always` 读不到新值；`acmd41_cnt` 在 CS 沿被清零导致永远回不到第 3 次；边际推理调不动时应换显式计数。

↓抽象成规则

技能包条目：“永远不要在信号自己更新的沿上读它”；“复位的作用域要精确界定：跨帧状态 ≠ 帧内状态”。

↓写成可执行检查

判据：写 TB 时自问——这个 `@(posedge/negedge X)` 和 X 的驱动沿是同一个

吗？是则改成 `@(posedge clk); #1;` 或换一个沿。

6.4 边界如何确定

判据：每条技能都必须能回答三个问题——□ 它解决什么现象？□ 它的前提是什么？□ 前提不成立时会怎样？

反面例子（避免把特例当通例）：

× **不要写**：“BRAM 一定要加 `(* ram_style = "block" *)`”——实测该属性在“1 写 2 读跨时钟”下**无效**。

✓ **应该写**：“BRAM 推断由**访问模式**决定，属性只是建议。1W1R 可靠；1W2R 跨时钟必退化成 LUTRAM，属性也救不了。”

7 复现说明

7.1 环境要求

表 16　软件环境

工具	版本	说明
Vivado	2020.2	免费 WebPACK 版即可（7A100T 在支持列表内）
iverilog	12.0	`pacman -S iverilog` / `apt install iverilog`
Verilator	5.052	`pacman -S verilator`
Verible	latest	`mason` 或 GitHub release
Python	≥3.10	系统自带
Pillow	≥10.0	仅 `mkimage.py` 处理 PNG/JPG 时需要
中文字体	Noto CJK	编译本文档需要

7.2 从零开始的完整步骤

7.2.1 步骤 1：获取代码

```
git clone https://github.com/flyingyuya/Sonic-Speed-Parallelism.git
cd Sonic-Speed-Parallelism
```

7.2.2 步骤 2：工具链自检

```
bash scripts/sim/run_iv.sh tools
# 预期：[ok] iverilog / verilator / verible 三行
# 若 Vivado 找不到：export PATH=$HOME/Xilinx/Vivado/2020.2/bin:$PATH
```

7.2.3 步骤 3：全量仿真（约 1 分 30 秒）

```
bash scripts/sim/run_iv.sh
# 预期：通过 41  失败 0  跳过 0     <-- 这是环境是否正确的黄金标准
# 单项：bash scripts/sim/run_iv.sh fft_core
# 含 tb_top 的全量：FULL=1 bash scripts/sim/run_iv.sh
```

7.2.4 步骤 4：静态检查

```
verible-verilog-lint --rules_config_search $(find rtl sim -name '*.v')
# 预期：无输出（退出码 0）
```

7.2.5 步骤 5：综合与实现（约 9~10 分钟）

```
vivado -mode batch -source scripts/vivado/build.tcl \
       -tclargs xc7a100tfgg484-2 top constrs
# 预期：构建 0  CRITICAL 0  WARNING 0
#       LUT 11716 (18.48%)  BRAM 100.5 (74.44%)
#       clk_sys +1.779ns    clk_hdmi +4.028ns
# 只综合（约 3 分钟，看资源够不够）：
SYNTH_ONLY=1 vivado -mode batch -source scripts/vivado/build.tcl \
       -tclargs xc7a100tfgg484-2 top constrs
```

7.2.6 步骤 6：准备 SD 卡（需要 sudo）

```
lsblk -o NAME,SIZE,TYPE,TRAN | grep -E "sd|mmc"     # 找到读卡器，如 sda

# 先 dry-run（强烈建议）
python3 scripts/sdcard/mkimage.py --dry-run /dev/sda img1.png img2.png img3.png
# 预期每张图 261120 字节 = 480x272x2  <-- 数字对了就说明格式正确

# 正式写入（脚本内建系统盘保护）
echo YES | sudo python3 scripts/sdcard/mkimage.py /dev/sda img1.png img2.png img3.png
```

7.2.7 步骤 7：上板

```
# 1) 验证 JTAG
vivado -mode batch -source scripts/vivado/program.tcl -tclargs detect
# 预期：发现 JTAG 目标 ... 认到目标器件 xc7a100t

```

```
# 2) 下载到 SRAM（10 秒，掉电即失，开发期用这个）
vivado -mode batch -source scripts/vivado/program.tcl

# 3) 固化到 Flash（43 秒，掉电保持，定型后才用）
vivado -mode batch -source scripts/vivado/program.tcl -tclargs flash
# 预期：Mfg ID ef / Erase-Program-Verify successful
# 注意：JTAG 优先于 QSPI，验证 Flash 启动必须拔掉 JTAG 线再上电
```

7.3 验收清单

表 17　上板验收清单

#	检查项	预期
1	`run_iv.sh`	**41/41 PASS**
2	`verible`	0 违规
3	构建	**0 CRITICAL / 0 WARNING**
4	`led[1]` 闪烁	FPGA 活着，时钟/复位正常
5	`led[0]` 常亮	WM8960 I2C 配置完成
6	LCD 显示	网格 + 渐变背景
7	按 BG 键	在程序化背景 / SD 图片间切换
8	按 SD 键	3 张图片循环
9	对麦克风说话	频谱柱跳动、波形变化
10	按 SPK 键	喇叭静音/取消
11	HDMI 显示器	640×360 有画面（80 柱 + 波形 + 图片）
12	UART 115200	上电 50 ms 打横幅；发 V 切视图

7.4 常见问题

表 18　常见问题与解决

现象	原因	解决
`run_iv.sh` 全失败	iverilog 太老	需 ≥11.0
综合报 `UTLZ-1`（LUTRAM 超限）	存储器多端口访问模式不匹配推断模板	复制实例，不要加读口
`place_design` 失败但无时序错	BRAM >95% 导致布线拥塞	看 `post_route_util.rpt`，降位深/降分辨率
上板 LCD 全白/全黑	LCD 时序参数错	跑 `run_iv.sh lcd_timing`
SD 图片乱码/斜切	卡里数据尺寸与 RTL 的 W/H 不匹配	重新写卡，确认每图 261 120 字节
HDMI 颜色偏黄	R/B 通道互换	见 5.4 例 5
HDMI 复位后偶发右半屏撕裂	**已知未解决**（TMDS 层）	重按复位可恢复；需 ILA/示波器
线路输入无声	板上 `C4`(1 μF) 开路	需补焊（代码已就绪）

附录 A　关键指标汇总

表 19　关键指标

类别	指标	值
规模	RTL 文件 / 行数	60 / ≈12 000
	testbench 数量	41
	问题账本条目	825
资源	Slice LUTs	11 716（18.48%）
	Block RAM	100.5（74.44%）
	DSP48	26（10.83%）
时序	`clk_sys` 48 MHz	WNS +1.779 ns
	`clk_hdmi` 25 MHz	WNS +4.028 ns
	`clk_pix` 12.5 MHz	WNS 远大于 0
质量	构建警告	0 CRITICAL / 0 WARNING
	静态检查	Verible 0 / Verilator 0
	仿真回归	41/41 PASS
性能	音频采样	48 kHz / 24 bit
	FFT 整帧	0.64 ms（占帧周期 1.3%）
	端到端延迟	≈82 ms（其中 80 ms 是 LCD 一帧扫描）
功能	LCD	480×272，8 按钮 + 8 行状态栏 + 双背景 + SD 图片
	HDMI	640×360，80 根独立柱 + 波形 + SD 图片（裁剪 2× 放大）
	音频	话筒 / EQ 5 段 / 双声道 / 静音 / 回环
	存储	SD（CMD18 多块读）/ QSPI Flash 掉电自启动

附录 B　与商业方案的对比

表 20　与商业方案对比

维度	PC 软件	Zynq+PYNQ	商用 ASIC	**本作品**
端到端延迟	10~100 ms	1~10 ms	<1 ms	**≈82 ms**
处理链路延迟	不确定	依赖 OS/缓存	确定	**<2 ms，可精确到拍**
在线调参	灵活	灵活	不可	**1 拍生效**
启动时间	秒级	秒级	即时	**微秒级**
IP 依赖	无	HLS+AXI	——	**零**
可审计性	高	低（HLS 不可见）	无	**每一行可查**
成本	低	中	高	**低**